SmartBridge ServiceNow Project Report

Detailed 8-Phase Documentation & Technical Guide

Project Title	**Implement Client Script and UI Policy (Incident)**
Platform & Module	ServiceNow ITSM (Information Technology Service Management)
Target Table	Incident Table (`incident`)
Submission Target	SmartBridge Project Review & SkillWallet Portal

1. EXECUTIVE SUMMARY & OVERVIEW

This technical report details the end-to-end design, configuration, and implementation of client-side operations on the **ServiceNow Incident table**. The primary aim of this project is to enhance data integrity, automate routine UI validation tasks, and improve overall user experience for Service Desk agents using a hybrid approach of custom **Client Scripts** (JavaScript) and declarative **UI Policies** (No-Code rules).

By enforcing strict data validation rules on critical fields like *Category*, *Short Description*, *Close Code*, and *Close Notes*, the ServiceNow instance guarantees high-quality ticket submission and seamless incident resolution tracking.

2. GITHUB PHASE REPOSITORY STRUCTURE

As specified in the SmartBridge submission instructions, the GitHub public repository is structured phase-wise into 8 distinct directory modules:

```
smartbridge-servicenow-incident-project/
├── 01_Brainstorming_and_Ideation/
│   └── Problem_Statement_and_Ideation.md
├── 02_Requirement_Analysis/
│   └── Functional_and_NonFunctional_Requirements.md
├── 03_Project_Design/
│   └── Architecture_and_Workflow_Diagrams.md
├── 04_Project_Planning/
│   └── Development_Schedule_and_WBS.md
├── 05_Project_Development/
│   ├── ClientScripts/Category_Software_Handler.js
│   └── UIPolicies/Resolution_Fields_Policy.json
├── 06_Project_Testing/
│   └── Test_Cases_and_Execution_Report.md
├── 07_Project_Documentation/
│   └── Technical_User_Guide.md
└── 08_Project_Demonstration/
    └── Video_Presentation_Link.md
```

Phase 1: Brainstorming & Ideation Phase

- **Identified Challenge:** Support staff often submit incidents with generic information, missing crucial diagnostic data, or resolve incidents without supplying standardized resolution codes. This leads to poor reporting quality and incomplete incident management logs.
- **Proposed Solution:** Utilize ServiceNow's Client Scripting API to dynamically manipulate forms based on user choices in real time, combined with UI Policies to enforce conditional read-only, mandatory, and visibility constraints without additional server calls.
- **Expected Impact:** Minimizes user error by 40%, ensures 100% compliance on mandatory resolution inputs, and reduces ticket resolution processing times.

Phase 2: Requirement Analysis Phase

Requirements were gathered and categorized into functional logic constraints and non-functional system benchmarks:

Requirement Type	Target Field	Specification / Condition
Functional (FR-01)	`category`	When changed to **Software**, automatically set `short_description` as **Mandatory**.
Functional (FR-02)	`description`	Auto-inject a structured ticket template into `description` if it is currently blank when Category is Software.
Functional (FR-03)	`close_code` & `close_notes`	When incident `state` is changed to **Resolved**, make both resolution fields **Mandatory** and **Visible**.
Non-Functional (NFR-01)	Form Latency	Client Script execution time must be under 50ms to prevent visual delay during form interactions.
Non-Functional (NFR-02)	Maintainability	Use UI Policies over JavaScript for field visibility where possible to adhere to ServiceNow Best Practices.

Phase 3: Project Design Phase

Workflow & Execution Order Design:

1. **Form Initialization:** User navigates to ServiceNow Incident form (`incident.do`).
2. **Client Script Evaluation:**
    - User selects `Category = Software`.
    - System triggers `onChange` Client Script.
    - Script evaluates `newValue`, sets `g_form.setMandatory('short_description', true)`, inserts template text, and displays informational message using `g_form.showFieldMsg()`.
3. **UI Policy Engine Evaluation:**
    - User changes Incident `State = Resolved (6)`.
    - UI Policy Engine detects state change and checks defined conditions.
    - UI Policy Actions execute dynamically: `close_code` and `close_notes` set to **Mandatory = True** & **Visible = True**.
    - If state reverts away from Resolved, **Reverse if false** automatically hides and un-mandates resolution fields.

Phase 4: Project Planning Phase

The project execution schedule was organized across 5 distinct milestones:

Timeline	Milestone	Deliverables & Tasks
Day 1	Environment Setup	

Timeline	Milestone	Deliverables & Tasks
		Provision ServiceNow Personal Developer Instance (PDI), configure admin roles, and create git repository.
Day 2	Client Scripting	Write, test, and debug `onChange` JavaScript logic on Incident table.
Day 3	UI Policy Config	Create UI Policy conditions and configure UI Policy Actions for Resolution fields.
Day 4	Quality Assurance	Execute positive and negative test scenarios; verify form responsiveness and cross-browser support.
Day 5	Submission Prep	Finalize GitHub phase documentation, export update sets, and record public video demonstration.

Phase 5: Project Development Phase (Implementation Code & Config)

1. ServiceNow Client Script Configuration:

Script Name	Incident - Category Software Handler
Table	Incident (`incident`)
UI Type	All (Desktop & Mobile/Service Portal)
Type & Field	onChange | Field: `category`

```
/**
 * ServiceNow Client Script - Category Software Handler
 * Table: Incident [incident]
 * Type: onChange (Field: Category)
 */
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    // Exit if form is loading or field value is cleared
    if (isLoading || newValue === '') {
        return;
    }

    // Check if selected category is 'software'
    if (newValue === 'software') {
        // Enforce Short Description as Mandatory
        g_form.setMandatory('short_description', true);

        // Populate structured template into Description field if empty
        if (g_form.getValue('description') === '') {
            var templateText = "=== SOFTWARE INCIDENT DETAILS ===
" +
                               "1. Application Name:
" +
                               "2. Error Message/Code:
" +
                               "3. Impacted Users Count:
" +
                               "4. Steps to Reproduce:";
            g_form.setValue('description', templateText);
        }

        // Show guidance message on form
        g_form.showFieldMsg('category', 'Please ensure application error logs are attached.',
'info');
    } else {
        // Reset mandatory attribute and clear field messages for other categories
        g_form.setMandatory('short_description', false);
        g_form.hideFieldMsg('category');
    }
}
```

2. ServiceNow UI Policy Configuration:

Policy Name	Make Close Info Mandatory on Resolved
Table	Incident (`incident`)
Conditions	`State` `is` `Resolved` (Value: 6)
Settings	Global: **True** | On load: **True** | Reverse if false: **True**

UI Policy Actions Configured:

Field Name	Mandatory	Visible	Read Only
`close_code` (Close code)	True	True	False
`close_notes` (Close notes)	True	True	False

Phase 6: Project Testing Phase

Comprehensive test cases were executed on a ServiceNow Washington/Utah developer instance to validate logic accuracy:

Test ID	Test Scenario	Execution Steps	Expected Outcome	Result
TC-01	Software Category Selected	1. Open new Incident form. 2. Set Category to 'Software'.	Short Description becomes mandatory; Template auto-fills in Description; Info message appears.	**PASS**
TC-02	Category Changed Away	1. Select 'Software'. 2. Change Category to 'Hardware'.	Short Description mandatory flag is removed; Info message disappears instantly.	**PASS**
TC-03	Incident State set to Resolved	1. Fill basic ticket info. 2. Change State to 'Resolved'.	Close Code and Close Notes appear on form and become mandatory.	**PASS**
TC-04	Incident State Reverted	1. Set State to 'Resolved'. 2. Change State to 'In Progress'.	Close fields are automatically hidden and reset to non-mandatory via UI Policy.	**PASS**

Phase 7: Project Documentation Phase

Best Practices & Optimization Summary:

- **Performance Optimization:** Included `if (isLoading || newValue === '') return;` early exit condition in Client Scripts to avoid performance lag during initial form load.
- **ServiceNow Architecture Guidelines:** Used UI Policies for field visibility/mandatory logic instead of writing complex DOM manipulations or heavy Client Scripts.
- **User Experience (UX):** Provided clear field messages (`showFieldMsg`) to guide service agents on exact ticket requirements.

Phase 8: Project Demonstration Phase (Video Presentation Script)

Demo Script for Google Drive Submission Video:

*"Hello everyone! Welcome to my project demonstration. My name is [Student Name], and today I am presenting our SmartBridge project titled **'Implement Client Script and UI Policy on Incident Table in ServiceNow'**.*

***Purpose & Objectives:** The main objective of this project is to streamline incident management, automate field validation, and ensure data completeness when IT support teams log and resolve incidents.*

Live Execution Walkthrough:
*1. **Client Script in Action:** As you can see on screen, when I set the Incident Category to 'Software', our client script immediately executes. The 'Short Description' field becomes mandatory with a red indicator, and a pre-formatted template is automatically populated in the 'Description' box.*
*2. **UI Policy in Action:** Next, when I move the incident state to 'Resolved', our UI Policy triggers. It automatically displays 'Close Code' and 'Close Notes' fields and makes them strictly mandatory before saving.*

***Final Output:** All 8 phases have been documented and committed phase-wise to our public GitHub repository. Thank you!"*